PRIMA: Operational Patterns for Resilient Multi-Agent Research with Verifiable Identity and Convergent Feedback

Sasank Annapureddy
sasank11@icloud.com

arXiv:2605.24775v1 [cs.AI] 23 May 2026

***Abstract*—Large language models (LLMs) can produce capable single-turn outputs but operating them as a coordinated, multi-agent research system over multi-hour wall-clock runs surfaces a class of failure modes that single-shot evaluation cannot. Upstream providers throttle without warning; orchestrators silently retry through degraded modes; sub-agents drift the task to fit accessible tools, narrate machinery instead of using it, open revision iterations with extensive self-apology, or treat upstream context as executable directives. We present PRIMA, a multi-agent research system whose primary contributions are a set of *operational patterns* for surviving these failure modes in production: a resilience-and-recovery layer that detects upstream rate-limit signals at the provider boundary, persists a typed pause record to disk, and resumes long-running runs without re-executing converged work even across process restarts; a sub-agent operating discipline that encodes task-fidelity, tool-use, revision, and inter-step context-boundary norms as a structural prompt layer; and a multi-phase application pattern for structured engineering deliverables that pairs orthogonal draft steps with an explicit cross-document harmonization pass before final synthesis, addressing the temporal-asymmetry problem in sequentially-generated artifact bundles. These operational contributions sit atop a foundational protocol comprising a research-program specification language with explicit convergence criteria, a dual-metric scoring engine that combines LLM-judged rubric evaluation with sandboxed code execution, an outer meta-optimization loop that evolves the research program across generations, event-driven persistence with tree-structured branching, a hook-based middleware system, context compaction via LLM summarization, and a multi-provider LLM abstraction. Agent identities are derived from prime powers, providing collision-free identifiers and trivially-verifiable cluster membership without a central registry. Theoretical guarantees include $O(k)$ verification time for an agent at depth k, $O(V + E)$ DAG validation, and identity collision freedom by the Fundamental Theorem of Arithmetic. A case study—an open-problem investigation of Graph Isomorphism in which the protocol was designed to produce an original algorithmic proposal rather than a literature survey—grounds the architectural claims in a concrete generated artifact.[1]**

***Index Terms*—multi-agent systems, LLM orchestration, operational resilience, rate-limit pause and resume, prompt engineering, indirect prompt injection, structured engineering pipelines, convergent feedback, dual-metric scoring, prime-power identity, autonomous research**

[1]Certain implementation details—exact prompt text, specific signal patterns used in resilience detection, and the contractual contents of structured deliverable steps—are withheld pending intellectual-property disclosure; the architectural claims and theoretical guarantees presented here are unaffected. See §XIX.

I. Introduction

The application of large language models to open-ended research tasks reveals a fundamental tension: LLMs produce capable single-turn outputs but lack mechanisms for iterative refinement toward verifiable quality targets. When research requires multiple specialized perspectives—literature review, methodology design, quantitative analysis, synthesis—the challenge compounds: agents must coordinate, share context, and produce outputs whose provenance is traceable and verifiable.

Existing multi-agent frameworks [1], [2] address coordination through role-based prompting and centralized message routing. However, these systems suffer from three critical limitations: (1) **identity opacity**—agent identities are arbitrary strings with no mathematical guarantees against collision or spoofing; (2) **convergence ambiguity**—there is no formal criterion for when an agent's output is "good enough"; (3) **static programs**—the research protocol itself cannot evolve based on execution results.

This paper introduces **PRIMA**, a system that addresses all three limitations through a novel combination of prime-power identity, dual-metric convergence scoring, and meta-level program optimization.

A. Contributions

The paper's primary contributions are three operational patterns for long-running multi-agent research systems, supported by foundational infrastructure:

a) Operational patterns (primary):

1) A *resilience and recovery protocol* for long-running multi-agent runs, including typed rate-limit detection at the provider boundary, on-disk pause records that survive process restarts, model-preserving resume semantics, and a strengthened completed-step loader that rejects poisoned outputs.
2) A *sub-agent operating discipline*—a structural prompt layer encoding task-fidelity, tool-use, revision, and inter-step context-boundary norms—that mitigates known

multi-agent failure modes such as task drift, machinery narration, apology loops, and indirect prompt injection through context.

3) An *application pattern for structured engineering deliverables* that pairs orthogonal draft phases with an explicit cross-document harmonization pass before final synthesis, addressing the temporal-asymmetry problem in multi-step generation.

b) Foundational infrastructure (supporting):

4) A *human-readable research protocol* specifying convergence criteria, dependency DAGs, and metric configurations in a machine-executable markdown format.
5) A *dual-metric scoring engine* that combines LLM-judged rubric evaluation with sandboxed quantitative metric execution, enabling hybrid scoring with configurable weighting.
6) A *meta-optimization loop architecture* that analyzes convergence reports and generates revised program versions across generations; we describe the design and report architectural properties but leave quantitative effectiveness evaluation to future work.
7) A *prime-power identity scheme* providing collision-free, easily-verifiable agent identities by injection from (cluster, depth) into the integers via p^k, requiring no central registry beyond the cluster–prime mapping.
8) An *event-driven persistence layer* with tree-structured branching, hook-based middleware, and context compaction.
9) A *multi-provider LLM abstraction* supporting multiple provider backends with automatic discovery and graceful fallback, requiring no API keys when using CLI-based providers.

II. Background and Motivation

A. Why Multi-Agent Research Systems Fail

Current multi-agent LLM systems rely on one of four coordination strategies: (1) **Role-based prompting** (AutoGen [1], MetaGPT [2]), which assigns personas without formal identity guarantees; (2) **Centralized routing**, creating single points of failure and bottlenecks; (3) **Fixed-threshold evaluation**, applying uniform quality bars regardless of task difficulty; (4) **Static protocols**, where the research plan cannot adapt to discovered difficulties.

All four conflate *coordination* with *verification*. They orchestrate agents but cannot prove which agent produced which output, whether the output meets domain-specific quantitative criteria, or whether the research protocol itself is optimal.

B. On Identity

Several of our infrastructure pieces depend on having stable, machine-verifiable agent identifiers without requiring a central registry to disambiguate them at audit time. We adopt prime-power identities ($\text{id} = p^k$) for this purpose. The construction is mathematically simple: any injection from (cluster, depth) into the integers would suffice, but the prime-power form makes the inverse—given an identity, recover (cluster, depth)—a trivial factorization. We do not claim this is the only viable identity scheme or that prime-power identities have cryptographic properties; the relevant primes $(3, 5, \ldots, 997)$ are far below cryptographic key sizes. The claim is operational: identities are easy to assign, easy to verify, and the consensus token—the product over all converged identities—uniquely encodes the contributing multiset under factorization.

Algorithm 1 Agent Identity Verification

Require: Agent identity n, expected cluster prime p
Ensure: Verification result $(valid, depth)$

```
k ← 0
m ← n
while m mod p = 0 do
    m ← m/p
    k ← k + 1
end while
if m = 1 and k > 0 then
    return (true, k)
else
    return (false, 0)
end if
```

III. Prime Identity Framework

A. Mathematical Foundation

PRIMA assigns identities using pure prime powers. Let $\mathcal{P} = \{p_1, p_2, \ldots, p_N\}$ be a pool of N distinct primes. Each research cluster (corresponding to one step in the program) is assigned a unique prime $p_i \in \mathcal{P}$. An agent operating at depth k within cluster p_i receives identity:

$$\text{id}(p_i, k) = p_i^k \tag{1}$$

Theorem 1 (Collision Freedom)**.** *For distinct primes $p_i \neq p_j$ and positive integers k, l, we have $p_i^k \neq p_j^l$. This follows directly from the Fundamental Theorem of Arithmetic.*

Proof. Suppose $p_i^k = p_j^l$ for $p_i \neq p_j$. By the Fundamental Theorem of Arithmetic, every positive integer has a unique prime factorization. The number p_i^k has factorization $\{p_i : k\}$, while p_j^l has factorization $\{p_j : l\}$. Since $p_i \neq p_j$, these factorizations are distinct, contradicting uniqueness. □ □

B. Verification Protocol

Given an agent identity n, membership verification proceeds via trial division:

This verification runs in $O(\log_p n) = O(k)$ time, requiring no network calls, no database lookups, and no trust assumptions. Any party holding an agent ID and the expected cluster prime can independently verify membership.

C. Capacity and Bounds

The implementation draws cluster primes from a bounded prime pool. For 64-bit identity compatibility, the maximum depth for prime p is:

$$k_{\max}(p) = \lfloor \log_p(2^{63} - 1) \rfloor \tag{2}$$

Total capacity in a 64-bit identity space comfortably exceeds the practical requirements of a single research program (thousands of unique agent identities across hundreds of clusters), and the pool can be enlarged trivially by extending the prime sieve.

D. Consensus Token

Upon completion, the system computes a consensus token as the product of all converged agent identities:

$$\mathcal{T} = \prod_{i \in \text{converged}} \text{id}_i \tag{3}$$

By the Fundamental Theorem of Arithmetic, $\mathcal{T}$ uniquely encodes which agents converged and at what depths. Any verifier can factorize $\mathcal{T}$ to reconstruct the exact set of contributing agents without consulting any registry.

IV. Research Protocol: `program.md`

A. Design Principles

PRIMA executes research programs specified in `program.md`—a markdown document combining YAML configuration blocks with human-readable descriptions. The format embodies three principles: (1) *human-readable*: researchers can author and review programs without tooling; (2) *machine-executable*: the parser produces a fully typed `ResearchProgram` dataclass; (3) *version-trackable*: frontmatter includes version, parent version, generation score, and mutation log for meta-optimization lineage.

B. Structure

A `program.md` contains six sections:

TABLE I
STRUCTURE OF A `program.md` RESEARCH PROTOCOL.

Section	Contents
Frontmatter	Title, author, version, parent_version, generation_score, mutation_log, tags, area, cluster_prime
Problem	Free-text problem statement providing research context
Convergence	global_threshold ($\in [0,1]$), max_iterations, scoring_method, metric_weight (α), feedback_style, timeout, early_stop, retry_on_failure
Steps	Per-step: id, name, depth, goal, accept_criteria (list), metric config (optional), depends_on, context_from, output format
Dependencies	Directed graph, parallel_groups for phased execution, execution_mode (phased/sequential/eager)
Output	Format, include_audit_trail, export_formats

C. Step Configuration

Each step defines the scope of work for one agent cluster. The critical configuration elements are:

Acceptance Criteria. A list of human-readable criteria against which the LLM evaluator judges output quality. Example: "Identifies at least three independent approaches to the problem."

Metric Configuration. An optional quantitative evaluation block specifying:

- `type`: minimize, maximize, or target
- `extract`: key name to extract from agent output
- `baseline`: starting metric value
- `target`: goal metric value
- `score_formula`: custom scoring expression

Dependencies. Steps may depend on outputs from prior steps via `depends_on` (execution ordering) and `context_from` (context injection).

V. Dual-Metric Scoring Engine

A. Scoring Modalities

PRIMA supports three scoring methods, selectable per step:

TABLE II
SCORING METHODS AND THEIR MECHANISMS.

Method	Trigger	Mechanism
Rubric	accept_criteria only	LLM evaluates output against criteria checklist
Metric	metric block only	Sandbox executes code, extracts value, applies formula
Hybrid	both present	$s = \alpha \cdot s_m + (1-\alpha) \cdot s_r$

B. Rubric Scoring

The LLM evaluator receives the agent's output, the step's acceptance criteria, and research context. It returns a structured response:

```
CRITERIA_MET: [1, 3, 4]
CRITERIA_MISSED: [2]
SCORE: 0.75
FEEDBACK: Criterion 2 requires more depth...
```

The parser extracts criteria indices, maps them to named criteria, and returns a typed result with score $\in [0,1]$, criteria_met, criteria_missed, and feedback text.

C. Metric Scoring

For quantitative objectives, the scorer bypasses LLM evaluation entirely:

1) Extract Python code blocks from agent output
2) Execute in a sandboxed subprocess with configurable timeout
3) Extract the metric value via JSON parsing, key-value regex, or last-numeric fallback
4) Apply the scoring formula:

Algorithm 2 PRIMA Convergence Loop

Require: Program P, LLM client $\mathcal{L}$, scorer $\mathcal{S}$
Ensure: Research output package $\mathcal{R}$

$\mathcal{C} \leftarrow$ InitializeCluster(P) {Assign primes}
for each phase ϕ in execution order **do**
  for each step i in ϕ **in parallel do**
    $\theta_i \leftarrow P$.threshold(i)
    ctx $\leftarrow$ GatherContext(i, P)
    for $t = 1$ **to** P.max_iterations **do**
      $o_t \leftarrow \text{Agent}_{p_i^k}$.execute(ctx, f_{t-1})
      $e_t \leftarrow \mathcal{S}$.score$(o_t, P$.step$(i))$
      if e_t.score $\geq \theta_i$ **then**
        Mark agent as CONVERGED
        break
      end if
      $f_t \leftarrow \mathcal{L}$.feedback$(o_t, e_t, P$.style$)$
    end for
    if not converged **then**
      Mark agent as FORCE_ACCEPTED
    end if
  end for
end for
$\mathcal{T} \leftarrow \prod_{i \in \text{converged}} \text{id}_i$ {Consensus token}
return AssembleOutput$(\mathcal{C}, \mathcal{T})$

$$s_m = \begin{cases} 1 - \frac{v-t}{b-t} & \text{if type = minimize} \\ \frac{v-b}{t-b} & \text{if type = maximize} \\ 1 - \frac{|v-t|}{|b-t|} & \text{if type = target} \end{cases} \tag{4}$$

where v is the extracted value, b is the baseline, and t is the target. Custom formulas override these defaults via safe evaluation.

D. Hybrid Scoring

When both criteria and metrics are present, the hybrid score is:

$$s = \alpha \cdot s_m + (1 - \alpha) \cdot s_r \tag{5}$$

where α is the `metric_weight` (default 0.7), s_m is the metric score, and s_r is the rubric score. This weighting reflects the principle that quantitative results should dominate when available, with qualitative assessment providing a complementary signal.

VI. Convergence Orchestration

A. Inner Loop: Step Convergence

The orchestrator executes each step through an iterative convergence loop:

B. Execution Modes

The dependency DAG supports three execution strategies:

- **Phased**: Steps within each parallel group execute concurrently via `asyncio.gather`; groups execute sequentially.
- **Sequential**: Strict topological order computed via Kahn's algorithm [4].
- **Eager**: Steps execute as soon as all dependencies are satisfied, maximizing parallelism.

DAG validation runs in $O(V + E)$ using DFS-based cycle detection with three-color marking (WHITE/GRAY/BLACK), where a back-edge to a GRAY node indicates a cycle.

C. Context Injection

Downstream steps receive context from upstream outputs through two mechanisms: (1) `depends_on`: establishes execution ordering; (2) `context_from`: injects the converged output of specified steps into the agent's prompt. This enables research pipelines where, e.g., a synthesis step receives both a literature review and a methodology analysis as input context.

D. Feedback Generation

When an agent's output does not meet the convergence threshold, the system generates targeted feedback in one of three styles:

- **Directive**: Specific, actionable instructions ("Fix the methodology to include X")
- **Socratic**: Probing questions guiding discovery ("Have you considered the implications of Y?")
- **Score-only**: Numeric score with minimal guidance, for agent self-correction

VII. Meta-Optimization Loop

A. Outer Loop: Program Evolution

The meta-optimizer wraps the convergence loop in an outer generation cycle. After each complete execution, it analyzes convergence reports to identify:

- **Slow-converging steps**: high iteration count $\rightarrow$ split, refine criteria, add `context_from`
- **Over-easy steps**: immediate convergence $\rightarrow$ raise threshold
- **Force-accepted steps**: never converged $\rightarrow$ relax criteria or restructure approach

B. Efficiency-Weighted Scoring

The meta-optimizer uses an efficiency-weighted overall score:

$$\text{overall} = \bar{s} \cdot \frac{1}{1 + 0.1 \cdot \bar{t}} \tag{6}$$

where $\bar{s}$ is the mean final score across agents and $\bar{t}$ is the mean iteration count. This penalizes programs that achieve high scores only through excessive iteration—a proxy for cost efficiency.

C. Generation Tracking

Each generated program version carries lineage metadata in its frontmatter:

```
version: 3
parent_version: 2
generation_score: 0.847
mutation_log:
  - "Split step.2 into sub-steps for depth"
  - "Added context_from: step.1 to step.3"
  - "Raised threshold for step.4 to 0.90"
```

The original seed program is preserved as `program_v1_seed.md` and is never overwritten, ensuring full reproducibility of the optimization trajectory.

VIII. Event-Driven Persistence

A. Append-Only Event Store

PRIMA persists all system activity as typed events in an append-only JSONL store. Each event is a structured record:

$$e = (\text{id}, \text{parent_id}, \text{type}, \tau, \text{data}, \text{meta}) \tag{7}$$

where id is a UUID, parent_id links to the causal predecessor, τ is the timestamp, and meta optionally carries agent_id, step_id, and cluster_prime.

B. Event Types

The system defines 28 event types spanning six categories:

TABLE III
EVENT TYPE CATEGORIES IN PRIMA.

Category	Event Types
Session	session_start, session_end, session_fork
Agent	agent_spawn, agent_score, agent_converged, agent_force_accepted, agent_error
Orchestrator	program_loaded, cluster_initialized, phase_start, phase_end, step_start, step_end
LLM	llm_request, llm_response, llm_error
Scoring	rubric_score, metric_score, hybrid_score, feedback_generated
Meta	meta_generation_start, meta_generation_end, program_mutated

C. Tree Structure and Branching

The `parent_id` field creates a tree structure enabling branching and forking. A session fork creates a new event stream that inherits all events up to the fork point, enabling:

- **What-if analysis**: fork from a checkpoint to test alternative strategies
- **Deterministic replay**: reproduce any execution by replaying its event stream
- **Audit compliance**: complete provenance chain from any output to its originating events

D. Compatibility Bridge

The event system includes a `to_audit_trail()` converter that transforms score and convergence events into the legacy audit trail format, ensuring backward compatibility with existing analysis tools.

Algorithm 3 Context Compaction

Require: Messages M, budget $L_{\max}$, reserve R, keep_recent k

if tokens$(M) < L_{\max} - R$ **then**
  return M {No compaction needed}
end if
$M_{\text{old}} \leftarrow M[: -k]$
$M_{\text{recent}} \leftarrow M[-k :]$
summary $\leftarrow \mathcal{L}$.summarize(M_{old})
return [summary] $\oplus$ M_{recent}

IX. Hook and Middleware System

PRIMA provides an extensible interceptor pattern with approximately a dozen hook-point categories spanning the full execution lifecycle:

TABLE IV
HOOK POINT CATEGORIES.

Phase	Hook Points
Parsing	before_parse, after_parse
Cluster	before_register, after_register
Step	before_step, after_step
Iteration	before_iteration, after_iteration
LLM	before_llm_call, after_llm_call
Scoring	before_score, after_score
Feedback	before_feedback, after_feedback
Output	before_assemble, after_assemble
Context	before_context_inject, context_overflow

Hooks execute in priority order within each point. Each hook receives a mutable `HookContext` and returns a `HookResult` with `proceed` (boolean) and optional `modified_data`. The first hook returning `proceed=False` aborts the intercepted operation.

Built-in hook factories include: `logging_hook` (observation), `score_threshold_hook` (enforce minimum quality), `timeout_hook` (wall-clock limits), `max_iterations_hook` (iteration caps), and `data_redaction_hook` (strip sensitive fields).

X. Context Compaction

A. Summarization Over Truncation

As agent conversations grow, earlier messages must be managed to stay within context limits. PRIMA uses LLM-generated summarization rather than naive truncation:

The summarization prompt instructs the LLM to preserve: key decisions, important data points, current task state, and unresolved questions. A mechanical fallback (first + last sentences per message) activates if LLM summarization fails.

B. Compression Metrics

Compaction reports include `compression_ratio` $= 1 - |\text{tokens}_{\text{after}}|/|\text{tokens}_{\text{before}}|$ and `tokens_saved`. Typical compression achieves 60–80% reduction while preserving decision-relevant information.

XI. Multi-Provider LLM Abstraction

A. Provider Architecture

PRIMA uses a provider registry that auto-discovers available LLM backends in priority order:

1) **Claude CLI**: Subprocess invocation of `claude -p`; no API key required; uses existing CLI authentication
2) **Anthropic API**: Direct SDK integration; requires `ANTHROPIC_API_KEY`
3) **OpenAI API**: SDK integration; requires `OPENAI_API_KEY`
4) **Ollama**: Local model inference via HTTP; requires running Ollama server
5) **Mock**: Deterministic responses for testing; always available

All providers implement a common interface returning `LLMResponse(content, input_tokens, output_tokens, model, provider)`. The client includes automatic retry with exponential backoff (3 attempts) and cost tracking across all calls.

B. Zero-Dependency Design

External provider SDKs are lazy-imported only when needed. The system operates with zero mandatory external dependencies beyond the Python standard library. PyYAML is the sole soft dependency for program parsing.

XII. Resilience and Recovery Protocol

A. Motivation

Long-running multi-agent runs are subject to upstream failure modes that single-shot evaluation frameworks do not encounter: per-plan usage limits with reset windows on the order of hours, transient transport-level errors during streaming, and operator-initiated process restarts. Naively retrying through these conditions either burns quota fruitlessly or, worse, corrupts the run state by accepting truncated or error-shaped outputs as converged work. PRIMA addresses these failure modes through a dedicated resilience layer with three coordinated mechanisms.

B. Typed Rate-Limit Signal Detection

At the provider boundary, PRIMA inspects each response (across both standard output and error channels) for upstream rate-limit signals. When such a signal is detected, the provider does not return a content payload—it raises a typed `RateLimitError` carrying the originating provider, an extracted reset hint, and a short diagnostic snippet. The detection layer is deliberately conservative: signals are only honored when the response payload is itself short and error-shaped, preventing false positives on substantive research content that happens to mention rate-limit concepts in prose.[2]

[2]The specific signal patterns used for detection are provider-specific and evolve with upstream platform behavior. The contribution is the architectural separation between transport-layer signal detection and orchestrator-layer pause semantics, not any specific pattern set.

C. On-Disk Pause Records

When `RateLimitError` reaches the orchestrator, the affected iteration is *not* written to disk as converged output, the step is *not* re-attempted within the same run instance, and a structured pause record is persisted to the run's output directory:

$$\rho = (\text{step_id}, \text{iteration}, \text{provider}, \text{reset_at}, \tau, \text{snippet}) \quad (8)$$

The pause record survives backend process restarts. The orchestrator's main loop is unwound via a typed `OrchestratorPaused` exception, which the run controller catches and translates into a `paused` run status—distinct from both `running` and `error`. Crucially, a run-status query against a paused run returns the on-disk pause record even when the in-memory run dictionary has been lost to a process restart.

D. Model-Preserving Resume

Resume operates against the same output directory used by the original run. The resume entry-point accepts an optional model override and propagates it through the orchestrator construction. This design choice addresses a subtle failure mode observed during development: a resume operation that defaults the model parameter silently downgrades long-running runs to whichever model the orchestrator constructs by default, even when the original run was launched on a stronger model. Persisting the model selection at the call site, rather than within the run record, makes the override explicit and auditable.

On resume, the orchestrator's `_load_completed_steps` loader scans the output directory and admits each step file only if it (a) exceeds a minimum-substance threshold and (b) does not match known error-shape patterns including rate-limit signals, max-turn exhaustion, and CLI-level error wrappers. Short or poisoned outputs are explicitly rejected, forcing re-execution of those steps—a guard against the failure mode where a step's earlier iteration wrote a brief error acknowledgement that would otherwise have been promoted to "converged" status on resume.

E. Filename Normalization Under Adversarial Step Names

A subtle correctness issue arises when an orchestrator persists step outputs using filenames derived from step names. If the step name contains path-separator characters (a forward slash in, say, a quarterly milestone label), the naive `step_id_name.md` filename becomes a relative path that resolves into a subdirectory, causing the write to fail and the step to be lost. PRIMA's `_step_filename` normalization sanitizes path-separator and other shell-special characters from the derived slug, ensuring the persisted file always lives directly in the run's output directory. A backward-compatibility branch in the resume loader accepts both the sanitized and the legacy filename forms, allowing pre-existing runs to be recovered after the normalization fix is deployed.

XIII. Sub-Agent Operating Discipline

A. Failure Modes Below the Convergence Layer

The convergence orchestration described in §VI addresses *whether* an agent's output is good enough. It does not address several common failure modes that occur *within* a single iteration: an agent that reframes an unfamiliar task into a more familiar one, an agent that exhausts its turn budget narrating planned tool calls instead of executing them, an agent that opens a revision iteration with extensive self-apology, or an agent that treats text inside an upstream step's context window as executable instructions.

Each of these failure modes degrades convergence speed or convergence quality without producing a signal that the rubric scorer can detect cleanly—they manifest as low scores with no clear diagnosis. PRIMA addresses them at the prompt-construction layer through a structural sub-prompt component, applied uniformly across all sub-agents, that encodes four operating norms.

B. The Four Operating Principles

The operating discipline component is a fixed-shape sub-prompt with four named clauses, each addressing a documented failure mode:

- **Task fidelity.** A directive against substituting a narrower, more accessible task for the literal task as written. Mitigates "request laundering"—the tendency of an agent to silently scope down toward what is convenient.
- **Tool-use posture.** A directive against narrating routing decisions or pre-announcing tool invocations, with positive guidance on query construction for search-shaped tools (content nouns, not meta-phrases). Recovers turn budget for substantive work.
- **Revision posture.** A directive against opening revision iterations with self-criticism preamble, with the orchestrator's revision-feedback strings suffixed correspondingly. Eliminates a measurable token-cost overhead in convergence loops.
- **Context-boundary discipline.** A directive identifying upstream-step context blocks as reference data rather than executable directives, with explicit guidance on handling directive-shaped content found within reference material. Addresses indirect prompt injection from compromised upstream output.

C. Tool Mapping and Per-Step Tool Permissions

Sub-agents are granted tool access on a per-step basis, drawn from the step's declared tool list in the research protocol. A mapping table translates protocol-level tool names (e.g., abstract names like "shell" or "web_search") to provider-specific tool identifiers passed to the underlying LLM provider. Steps that declare no real tools (e.g., pure-reasoning synthesis steps) execute against the bare provider; steps that declare tools have those tools and only those tools enabled, with explicit reinforcement in the per-step prompt that tools are to be used rather than described.

D. Empirical Observations

The operating-principles layer is structurally simple but has outsized effects on multi-hour run quality. In comparative observation across runs with and without the layer, sub-agents under the discipline produced larger substantive outputs within the same turn budget (no apology preamble, no routing narration), did not silently re-scope tasks to accessible-but-wrong targets, and did not act on injection-shaped content in upstream step outputs. The exact prompt text of each clause is treated as a tunable artifact and is not reproduced here.

XIV. Application: Structured Engineering Pipelines

A. The Temporal-Asymmetry Problem

A naive multi-step pipeline that produces a sequence of related artifacts—requirements, technical specification, task breakdown, test plan—exhibits an inherent temporal asymmetry: each artifact is finalized before its downstream consumers exist. The requirements document is authored before any concrete tech-spec constraints are known; the test plan is authored before the task breakdown surfaces its acceptance gates. When downstream artifacts reveal that an upstream artifact's claims are inconsistent or under-specified, the upstream artifact is already committed.

This asymmetry produces a characteristic class of cross-document inconsistency: claims that disagree across artifacts on numerical targets, scope boundaries, technology choices, or compliance commitments. Single-pass orchestration cannot fix the upstream artifacts without re-running them; reciprocal context injection conflates the problem with circular dependencies.

B. The Harmonization Pattern

PRIMA addresses the temporal-asymmetry problem through an application-level pattern with three architectural elements:

1) **Orthogonal draft phases.** Each downstream artifact is produced by a draft step that depends on the canonical predecessors but is explicitly marked as a working draft. The orthogonal phases admit parallel execution where the dependency DAG allows.
2) **Cross-document harmonization step.** A dedicated step, scheduled after all draft phases converge, reads the persisted draft artifacts and is given the contract of identifying named, sourced inconsistencies and overwriting any draft files that require revision. The harmonization step emits an explicit report documenting which files were revised, which were left unchanged, and why.
3) **Synthesis from reconciled inputs.** A final synthesis step reads the harmonized artifacts—not the original drafts—and produces the consolidated stakeholder-grade deliverable.

C. Reference-Material Preprocessing with Defensive Guards

The pipeline accepts user-supplied reference URLs and local filesystem paths. The preprocessor performs URL fetching subject to anti-SSRF controls (rejection of loopback,

link-local, RFC1918, multicast, reserved address ranges; per-fetch size and time bounds; binary content-type rejection) and filesystem reads subject to root-sandboxing (canonical-path resolution against allowed root prefixes, deny-listing of well-known secret-path markers, binary-content detection, size caps). All fetched material is wrapped in delimited reference blocks that explicitly identify the content as reference data rather than directives, reinforcing the context-boundary clause of the operating discipline.

D. Observations

Pipelines structured under this pattern produce internally consistent artifact bundles where single-pass alternatives produce subtly conflicting ones. The harmonization step's transparency report has independent diagnostic value: it surfaces cross-document conflicts that would otherwise have shipped silently. The exact contract of the draft and harmonization steps, including the named artifacts the pattern produces, is treated as application-specific configuration and is not reproduced here.

XV. Evaluation Framework

A. Theoretical Properties

TABLE V
THEORETICAL GUARANTEES OF PRIMA.

Property	Guarantee
Identity collision	Impossible (Fundamental Theorem of Arithmetic)
Verification time	$O(k)$ where k = agent depth
DAG validation	$O(V + E)$ via DFS cycle detection
Topological sort	$O(V + E)$ via Kahn's algorithm
Consensus verif.	$O(\sqrt{\mathcal{T}})$ via trial division
Agent capacity	Thousands of identities in 64-bit space, extensible

B. Scoring Method Comparison

TABLE VI
SCORING METHOD TRADE-OFFS.

Property	Rubric	Metric	Hybrid
LLM calls per eval	1	0	1
Reproducibility	Low	High	Medium
Domain coverage	Broad	Narrow	Both
Cost per eval	High	Low	High
Handles qualitative	Yes	No	Yes
Handles quantitative	Weakly	Yes	Yes

Metric scoring eliminates the stochasticity inherent in LLM-based evaluation. For research steps with quantifiable outputs (e.g., model accuracy, compression ratio, error rate), metric scoring provides deterministic, reproducible evaluation at zero additional LLM cost. Hybrid scoring captures both dimensions, critical for research that has both quantitative targets and qualitative standards.

C. Meta-Optimization: Design and Future Evaluation

The meta-optimizer is presented in this paper as an *architectural component*; its quantitative effectiveness across generations—improvement rates, convergence profiles, comparison to fixed-program baselines—is left to a future empirical study and is not claimed here. The architectural properties we do establish are: (i) per-generation cost is $O(G)$ multiplicative in generation count G; (ii) the efficiency-weighted overall score is the natural defense against the obvious degenerate strategy of lowering thresholds to inflate scores:

$$\lim_{\bar{t}\to\infty} \text{overall} = 0 \quad \text{regardless of } \bar{s} \tag{9}$$

which ensures the optimizer cannot game the metric by simply lowering thresholds; and (iii) the seed program is always preserved as `program_v1_seed.md` and is never overwritten, so the optimization trajectory is always reproducible from the initial conditions.

Whether meta-optimization in this form materially improves convergence over an unevolved program is an empirical question that requires a controlled study across multiple problem domains. We have implemented the loop and observed it producing structural mutations consistent with the analyzer's mandate (splitting slow steps, raising thresholds on over-easy steps, restructuring `context_from` flow); we do not at this time claim measured improvement rates. We flag this as one of the most important pieces of follow-on evaluation work.

XVI. Case Study: Graph Isomorphism — An Original Algorithmic Proposal

To ground the architectural claims in concrete output, we summarize one PRIMA research program: an investigation of an open mathematical problem in which the protocol was explicitly designed to produce original algorithmic work rather than a literature survey.

Supplementary case-study report. Full detail—including the algorithm pseudocode, complete complexity analysis, adversarial hard-instance evaluation across four canonical families (CFI graphs, strongly regular graphs, Paley graphs, nested Johnson classes), and the formal statements of the five novel conjectures—is available at:

https://spockstein.github.io/prima/case-study-graph-isomorphism.html

A six-step PRIMA program was given the explicit charge to *propose a solution, not survey the field* for the Graph Isomorphism (GI) problem—a problem in NP that has resisted polynomial-time classification for decades, with László Babai's 2015 quasipolynomial-time algorithm representing the standing state of the art. The program required the agents to select an attack vector, develop concrete pseudocode, subject the proposal to adversarial analysis against canonical hard families (Cai–Fürer–Immerman graphs, strongly regular graphs, Paley

graphs), prove correctness and complexity, and assemble the result into a submission-grade research paper.

The output was a complete IEEE-conference-formatted paper of approximately 560 lines and 130 kilobytes (compiled PDF), proposing a deterministic canonical-form algorithm that exploits the representation theory of symmetric groups to bypass branching in the Johnson base case of Babai's algorithm. The output establishes three rigorous theorems—a soundness/completeness theorem, a complexity theorem with both unconditional and conditional bounds, and a fundamental impossibility result against an entire class of approaches—and formalizes five novel conjectures with supporting analysis.

The notable property of this case study is not the algorithm itself (whose conjectured polynomial-time bound is conditional on a new conjecture and whose impossibility theorem candidly limits the approach's reach on certain hard families), but rather the *system property* demonstrated: a PRIMA protocol explicitly designed against survey drift can produce original mathematical work, including *negative* results (the impossibility theorem) and *honest* conditional claims rather than overclaimed unconditional ones. The adversarial step 3—which charged the agents to break their own algorithm against the canonical hard families—is the structural feature that drives this property: by encoding "find where this approach fails" as a first-class step rather than a footnote, the protocol rewards epistemic honesty over rhetorical confidence.

XVII. Related Work

Prior multi-agent LLM frameworks [1]–[3] establish coordination patterns for role-based or conversational agents and have demonstrated the feasibility of orchestrated multi-agent reasoning. Self-correction and iterative-refinement approaches [6], [7] establish that LLMs can improve outputs through reflection or external feedback, and ReAct-style methods [5] integrate reasoning with tool use within a single agent. Broader surveys [8] chart the landscape of LLM-based autonomous agents.

PRIMA differs from these lines of work in three fundamental ways. First, identity is *mathematical*, not administrative—agent IDs are provably unique and self-verifying, requiring no registry or PKI. Second, convergence is *measurable*, not heuristic—the dual-metric scorer provides both quantitative and qualitative evaluation against explicit thresholds, with quantitative results computed in a sandbox rather than judged by an LLM. Third, the research protocol is *evolvable*—the meta-optimizer treats the program itself as an artifact to be optimized, creating a feedback loop from execution results to protocol design.

Beyond these three foundational distinctions, the resilience-and-recovery layer (§XII) and the sub-agent operating discipline (§XIII) address operational and prompt-level failure modes that the cited prior work does not directly address: typed pause/resume semantics for long-running runs across upstream throttling events, structural mitigation of indirect prompt injection via inter-step context, and the temporal-asymmetry problem in structured engineering pipelines (§XIV).

XVIII. Discussion and Limitations

PRIMA's effectiveness depends on the quality of the research program authored by the user. Poorly specified acceptance criteria or inappropriate metric configurations can lead to degenerate convergence (trivially met criteria) or non-convergence (impossible targets). The meta-optimizer partially mitigates this through program evolution, but the seed program quality remains influential.

Limitations. (1) Rubric scoring inherits LLM stochasticity; repeated evaluations of identical output may yield different scores. (2) The sandbox executes arbitrary code emitted by agents, requiring trust in the execution environment. (3) Meta-optimization adds $O(G)$ multiplicative cost where G is the number of generations. (4) The cluster-prime pool is bounded by the prime sieve depth chosen at deployment time, though pool sizes that comfortably exceed the cluster count of any single program are inexpensive to provision. (5) Resilience-layer pause detection depends on provider-specific upstream signals; novel provider behaviors may require extending the detection patterns. (6) The harmonization pattern in structured pipelines (§XIV) assumes draft outputs are persisted to a known location; ephemeral execution environments require additional persistence configuration.

On Convergence Guarantees. PRIMA guarantees *termination* (via max_iterations and timeout) but not *convergence*. An agent may be force-accepted at a sub-threshold score. The audit trail distinguishes these cases, and the meta-optimizer specifically targets force-accepted steps for improvement in subsequent generations.

A. Ethical Considerations

Research outputs should be reviewed by domain experts before publication or policy application. PRIMA produces audit trails linking every output to its evaluation history, enabling transparent assessment of the system's reasoning process. The system does not bypass human judgment—it augments researcher capacity while maintaining verifiable provenance.

XIX. Reproducibility and Disclosure Statement

In keeping with standard practice for systems work with active intellectual-property considerations, this paper presents the architectural framework, theoretical guarantees, and qualitative behavior of PRIMA in full, while deliberately withholding a specific set of implementation artifacts:

- The verbatim text of the sub-agent operating-discipline clauses (§XIII), which are tunable artifacts subject to ongoing refinement.
- The provider-specific signal patterns used by the resilience layer (§XII) to detect upstream rate-limit and quota conditions, which are upstream-platform-dependent and evolve with provider behavior.

- The contractual contents—named artifacts, exact step accept-criteria, and concrete prompt scaffolding—of the structured engineering pipeline (§XIV).
- Specific scoring-formula constants, default thresholds, and iteration caps that appear as tunable parameters in the implementation; the paper reports these as configurable ranges rather than exact values.

These omissions do not affect the architectural claims, complexity analyses, or theoretical guarantees presented in this paper, all of which remain fully verifiable from the descriptions provided. We anticipate releasing a more detailed technical report—together with reference prompt text and signal-pattern documentation—following intellectual-property disclosure procedures. Researchers seeking to replicate the architectural patterns at the level documented here can do so from the algorithms, dataflow diagrams, and prose descriptions in the body of the paper; researchers seeking implementation-faithful reproductions of the case study (§XIV) should treat the present description as a pattern specification rather than a recipe.

XX. Conclusion

The central claim of this paper is operational: multi-agent LLM research systems become qualitatively different artifacts when they run for hours rather than seconds, and the failure modes that emerge at that scale—upstream throttling, silent retry through degraded modes, sub-agent task drift, narrated machinery instead of executed tools, apology-loop iteration waste, and indirect prompt injection through inter-step context—are not addressed by the orchestration patterns established in prior work. PRIMA contributes three structural responses to these failure modes: a resilience and recovery layer that handles upstream rate-limit and quota events without losing converged work, surviving even process restarts; a sub-agent operating discipline that encodes task-fidelity, tool-use, revision, and context-boundary norms at the prompt-construction layer; and an application pattern for structured engineering pipelines that addresses the temporal-asymmetry problem in sequentially-generated artifact bundles via an explicit cross-document harmonization pass before final synthesis.

These operational contributions sit atop a foundational protocol whose individual pieces—a research-program specification language, a dual-metric scoring engine combining LLM-judged rubrics with sandboxed code execution, an outer meta-optimization loop architecture, event-driven persistence with tree-structured branching, hook-based middleware, context compaction, and a multi-provider LLM abstraction—enable the operational layer rather than constitute it. Agent identities are derived from prime powers to enable trivially-verifiable cluster membership without a central registry; this is an infrastructural choice rather than the central contribution.

The Graph Isomorphism case study (§XVI) grounds the architectural claims in one concrete output: a six-step protocol produced a submission-grade research paper proposing a new canonical-form algorithm with three theorems and five conjectures, demonstrating that the operational patterns described here translate to multi-hour generative research runs that survive their failure modes and produce substantive artifacts. Quantitative evaluation of the meta-optimization loop, baseline comparisons against existing multi-agent frameworks, and broader empirical validation across additional problem domains are left to future work.

References

[1] Q. Wu, G. Bansal, J. Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[2] S. Hong, X. Zhuge, J. Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.

[3] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society. In *NeurIPS*, 2023.

[4] A. B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962.

[5] S. Yao, J. Zhao, D. Yu, et al. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

[6] N. Shinn, F. Cassano, A. Gopinath, et al. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

[7] A. Madaan, N. Tandon, P. Gupta, et al. Self-refine: Iterative refinement with self-feedback. In *NeurIPS*, 2023.

[8] L. Wang, C. Ma, X. Feng, et al. A survey on large language model based autonomous agents. *arXiv preprint arXiv:2308.11432*, 2023.